



7. Altri algoritmi di sorting



Lezione 17 - @October 31, 2025

Ordinare senza confrontare

Problema

ordinare n interi in $[0 \dots b-1]$

array Y conta quante volte compaiono gli interi in A

ordinare n record cocon chiavi interi in $[0 \dots b-1]$

→ `bucketSort` implementazione con code.

Pseudo-codice

```
ALGORITMO bucketSort(Array  $A[0..n-1]$ , intero  $b$ )
  Sia  $Y[0..b-1]$  un array      //
  FOR  $i \leftarrow 0$  TO  $b-1$  DO    // predisposizione bucket
     $Y[i] \leftarrow \text{coda vuota}$     //
  FOR  $i \leftarrow 0$  TO  $n-1$  DO
     $x \leftarrow A[i].\text{chiave}$       //
     $Y[x].\text{enqueue}(A[i])$         // riempimento bucket
   $j \leftarrow 0$                   // posizione in cui devo riempire A
  FOR  $i \leftarrow 0$  TO  $b-1$  DO
    WHILE NOT  $Y[i].\text{isEmpty}()$  DO //
       $A[j] \leftarrow Y[i].\text{dequeue}()$  // riempimento array ordinato
       $j \leftarrow j + 1$            //
```



le operazioni sulla coda costano $O(1)$

- predisposizione bucket $\Theta(b)$
 - riempimento bucket $\Theta(n)$
 - riempimento array ordinato:
 - ciclo `for` $\Theta(b)$ iterazioni
 - ciclo `while` : complessivamente n passi
- $\Theta(b + n)$

Tempo totale

$$\Theta(b) + \Theta(n) + \Theta(b + n) = \Theta(b + n)$$

Se $b = O(n) \Rightarrow \Theta(b + n) = \Theta(n)$ tempo lineare!

Se $b \approx n^2 \Rightarrow \Theta(b + n) = \Theta(n^2)$ meglio usare `heapSort` !

Ricapitolando

- algoritmo non in loco

- **stabile**
 - utilizzabile su liste concatenate
-

Problema

ordinare un insieme di persone rispetto alla data del compleanno

bucket sort ordina la parte “meno significativa”, garantendo la stabilità permette di utilizzare meno code a parità di ordine in cui si applica l’algoritmo sulle chiavi scelte.

→ `radixSort`

fissata la base il numero di bucket è costante.

Radix sort

se devo ordinare tra 0 e 999’999’999:

prendo $b = 10^3$:

- 1000 bucket
- 3 passaggi di `bucketSort`

se devo ordinare n chiavi su 32 bit cioè tra 0 e $2^{32}-1$:

- divido il numero in 4 parti

→ $b = 2^8$ 256 bucket, 4 passate

→ $b = 2^{16}$ 65’636 bucket, 2 passate

Tempo $\Theta(n)$

come mai è “conveniente” usare la base 2?

- con la base 10 devo fare divisioni con resto e in modulo
es: $x = 1234 \rightarrow (x/10^2) \text{ MOD } 10$
in generale la formula: $(x/b^t) \text{ MOD } b$
- in base 2 vengono tradotte in `shift` che richiedono tempo costante.